AI Assistant for Service Businesses

Task 1: Articulate the problem and the user of your application

Problem Description

Service-based businesses face operational inefficiencies due to manual appointment scheduling, fragmented customer communication, and lack of automated availability management.

Target User and Why This is a Problem

The primary user of this application is a **small business owner or salon manager** (e.g., a hair salon owner) who is responsible for managing appointments, responding to customer inquiries, and ensuring smooth daily operations. In small service-based businesses, booking management is often handled manually through phone calls, social media messages, or messaging apps, which creates constant interruptions during working hours.

Manual scheduling increases the likelihood of double bookings, missed appointments, scheduling conflicts, and delayed responses to customer inquiries. Additionally, customers increasingly expect instant replies and the ability to book appointments outside of business hours. Without automation, business owners must either respond continuously throughout the day or risk losing potential clients.

This administrative burden reduces productivity, and limits the owner's ability to focus on delivering high-quality services. The goal of this system is to automate the booking workflow, reduce manual coordination, and provide customers with immediate, intelligent assistance through a conversational AI agent.

Potential User Questions and Evaluation Scenarios

1. Booking a Service

   Input:

   "I would like to book a haircut this Friday at 3 PM."

   Expected Output:

The system checks availability for the requested time, verifies that the time slot does not conflict with existing bookings, collects the user's phone number if necessary, and confirms the booking.

2. Checking Availability
   Input:

   "Do you have any available appointments tomorrow for coloring?"

   Expected Output:

   The system retrieves the duration of the requested service, calculates available time slots within working hours, filters out conflicting bookings, and returns valid options.

3. Returning Customer Request
   Input:

   "I want to book the same service as last time."

   Expected Output:

   The system identifies the customer via their phone number, retrieves their previous booking information, and suggests the same service before confirming availability.

4. Appointment Cancellation
   Input:

   "I need to cancel my appointment."

   Expected Output:

   The system retrieves the customer's active bookings, allows the user to select the relevant appointment, and updates the booking status according to cancellation rules.

5. Public Information Inquiry
   Input:

   "What are the most popular haircut trends this year?"

   Expected Output:

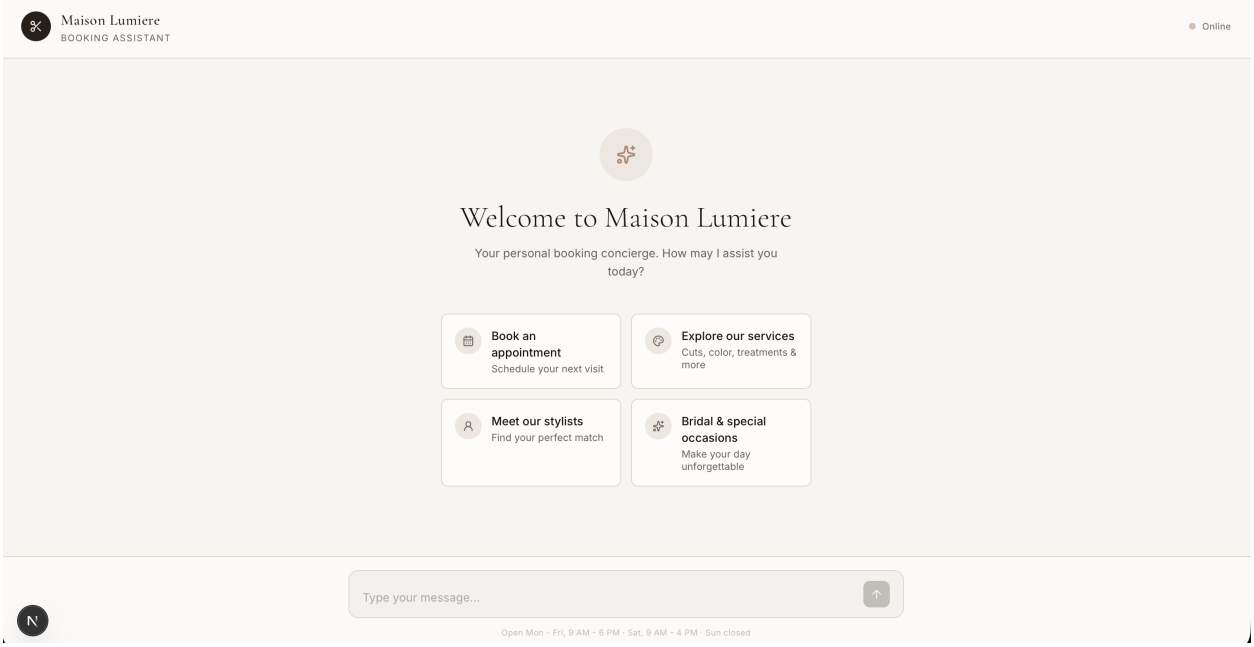
   The system recognizes that the question is not related to internal booking data, performs a public web search, and returns a summarized response based on publicly available information.

Task 2: Articulate your proposed solution

Proposed Solution Overview

To address the problem of manual appointment scheduling for small service-based businesses, I propose building an **AI-powered booking assistant** that combines Retrieval-Augmented Generation (RAG), tool-based agent orchestration, and structured backend services. In the future, possibly a **multi-tenant assistant.**

From the user's perspective, the solution appears as a conversational chat interface embedded on the business website. Customers can book appointments, check availability, cancel reservations, and ask service-related questions in natural language. The system responds instantly, verifies availability through backend validation, and enforces scheduling rules automatically. The business owner does not need to manually coordinate bookings, as the AI agent handles repetitive and rule-based tasks.



Technically, the solution follows the modern LLM application stack: a large language model acts as the reasoning engine, an agent orchestration layer manages tool usage, backend APIs enforce business logic, and a vector database enables context retrieval from internal documents such as pricing, services, and policies.

Infrastructure Diagram

User (Chat Interface)

↓

```
LLM Agent (Reasoning Layer)
      ↓
Agent Orchestration (LangChain)
      ↓
Tools Layer
  - Booking Tools
  - CRM Tools
  - Web Search Tool
      ↓
In-Memory Store (Now)  / Database (Future)
      ↓
Vector Database
      ↓
Embedding Model
```

Infrastructure diagram (in words)

- **User browser** → talks to **Next.js/React/Tailwind frontend** (chat UI).
- **Frontend** → calls the **FastAPI backend** at /chat//api/chat.
- **FastAPI backend** → calls the **LangGraph agent** from get_agent in agent.py.
- **LangGraph agent** → uses:
  - **OpenAI Chat LLM (gpt-4o-mini)** for reasoning and responding.
  - **LangChain tools**:
    - advanced_retrieve / retrieve → **Qdrant vector DB** via QdrantVectorStore + **OpenAI embeddings (text-embedding-3-small)**, optionally reranked with **Cohere rerank**.
    - get_available_slots, create_booking, list_bookings, cancel_booking, resc hedule_booking → operate on an **in-memory booking store** (InMemoryStore).
    - get_current_datetime → system time.
    - search_web_current → **Tavily Search API** for web RAG.
- **Ingestion notebook** (notebooks/ingest.ipynb) → reads data/data.txt, generates **embeddings** with OpenAI, and writes them into **Qdrant**.

- **Evaluation notebook** (notebooks/evaluation.ipynb) → uses **Ragas** personas and test sets to hit the agent and compute evaluation metrics.
- All secrets and endpoints are configured via **.env / .env.local** files for backend and frontend.

Tooling Choices

- **LLM(s)**: **OpenAI gpt-4o-mini** is used because it gives a good balance of quality, tool-usage reliability, and low latency/cost for a chat-style booking assistant.
- **Agent orchestration framework**: **LangGraph (with LangChain's create_agent)** is used because it makes it easy to define a tool-using, stateful agent with checkpoints and a clean separation between tools and orchestration.
- **Tool(s)**: Custom LangChain tools for retrieval, slot calculation, booking, current time, and web search were defined so that each real salon operation becomes an explicit, inspectable action the agent can invoke.
- **Embedding model**: **OpenAI text-embedding-3-small** is used for document embeddings because it's inexpensive, high-quality, and has first-class integration with LangChain and Qdrant.
- **Vector Database**: **Qdrant** is used as the vector store because it is fast, easy to run with an HTTP API, and has a native LangChain integration via QdrantVectorStore.
- **Monitoring tool**: For now, the project relies on **FastAPI/uvicorn logs and Qdrant's own observability** rather than a dedicated monitoring platform, which is sufficient for a small, local-first prototype.
- **Evaluation framework**: **Ragas** is used in the evaluation.ipynb notebook because it provides ready-made personas, testset generation, and RAG-specific quality metrics for systematically evaluating the assistant.

- **User interface**: **Next.js + React + Tailwind + Radix UI** are used because they allow a modern, responsive chat interface to be built quickly with good developer ergonomics and accessible components.
- **Deployment tool**: **uv + uvicorn for the backend and npm run dev for the frontend** are used because they keep local development and deployment simple, reproducible, and aligned with common Python/Node workflows.
- **Any other components**: **Tavily Search** is used for up-to-date web information, **Cohere rerank** sharpens retrieval quality over Qdrant results, and **LangGraph's InMemoryStore** is used to persist bookings in a simple way without bringing in an external database.

RAG and Agent Components

- **RAG components**:
  - Knowledge source: backend/data/data.txt (salon services, prices, hours).
  - Ingestion: notebooks/ingest.ipynb that embeds this text with OpenAI text-embedding-3-small and writes it into the Qdrant collection documents.
  - Retrieval: QdrantVectorStore + retriever/wide_retriever in agent.py, exposed as the retrieve and advanced_retrieve tools.
  - Reranking: optional **Cohere rerank v3.5** inside advanced_retrieve.
  - External RAG: search_web_current tool using **Tavily Search** for web-based augmentation when the local KB doesn't have the answer.
- **Agent components**:
  - Core agent: the LangGraph agent created in get_agent() using create_agent, with model="openai:gpt-4o-mini", MemorySaver checkpointing, and InMemoryStore as a state store.
  - Tools: get_current_datetime, retrieve, advanced_retrieve, get_available_slots, create_booking, list_bookings, cancel_booking, reschedule_booking, search_web_current.

- Business state:
  the InMemoryStore under the BOOKINGS_NS namespace where bookings are created, listed, cancelled, and rescheduled.
- API wrapper and UI: the FastAPI endpoints (/chat, /api/chat) that call agent.ainvoke(...) and the Next.js frontend that sends user messages and renders the agent's responses.

Task 3: Collect your own data (RAG) and choose at least one external API to use (Agent)

Likely user questions

- **Booking & availability**
  - "Can I book a haircut on Thursday after 5pm?"
  - "What times do you have available for a color treatment this Saturday?"
  - "Can you book me in for a haircut next week around lunchtime?"
- **Services & pricing**
  - "What services do you offer and how much do they cost?"
  - "How long does a balayage usually take?"
  - "Do you offer kids' haircuts or beard trims?"
- **Opening hours & policies**
  - "What are your opening hours?"
  - "Are you open on Sundays or public holidays?"
  - "What is your cancellation policy?"
- **Existing bookings (agentic tools over state)**
  - "Can you show me my upcoming appointment?"
  - "I need to cancel my appointment for tomorrow at 3pm."
  - "Can you move my booking from 10:00 to 11:30 instead?"
- **General styling / external info (Tavily)**

- “What haircut is trending for short curly hair right now?”
- “Is keratin treatment safe for dyed hair?”
- “What’s the difference between balayage and ombré?”

Default chunking strategy for my data

- **Chunking strategy**: I treat the salon knowledge base (data/data.txt) as a set of short, human-readable sections (e.g. services, prices, hours, policies) and chunk it at the **paragraph / logical section level** into relatively small text units (roughly a few sentences each), with **light overlap** between neighboring chunks. I use **LangChain’s RecursiveCharacterTextSplitter** with a **character-level** strategy over the full data.txt, configured with:
  - **Separators**: ["\n\n", "\n", " ", ""] so it prefers to break on paragraph and line boundaries before falling back to raw characters.
  - **chunk_size**: **800 characters** to keep each chunk small, focused, and cheap to embed while still usually containing a full service/price/hours description.
  - **chunk_overlap**: **200 characters** so that information that sits near a boundary (e.g. a duration at the end of one paragraph and a price at the start of the next) is still captured in at least one chunk.
- **Why this decision**:
  - The content is concise and domain-specific (services, durations, hours), so **small, semantically-coherent chunks** are enough to answer most questions without mixing unrelated topics.
  - Paragraph-level chunks reduce the chance of retrieving irrelevant information (e.g., another service’s price) while still giving the model all the fields it needs (name, duration, price) in one hit.
  - A little overlap helps with boundary cases (information spanning two lines/sections) without blowing up the index size.

Data source and external API, and how they interact

- **Data source (RAG side)**:
  - My primary data source is a **custom salon knowledge file** (data/data.txt) containing services, descriptions, durations, prices, opening hours, and simple policies.
  - This file is processed in the **ingestion notebook**, which:
    - Reads the raw text and splits it into the paragraph-level chunks described above.
    - Embeds each chunk using **OpenAI text-embedding-3-small**.
    - Stores embeddings and metadata into a **Qdrant** collection (documents), which is then queried at runtime via QdrantVectorStore and the retrieve / advanced_retrieve tools.
- **External API (agent side)**:
  - The main external "agentic" API is **Tavily Search**, wrapped in the search_web_current tool.
  - When the assistant needs up-to-date or more general information (e.g. hair trends, product safety, style ideas) that is not contained in the local salon KB, it calls Tavily through this tool to fetch **public web results**, which the LLM then summarizes and cites back to the user.
- **How they interact during usage**:
  - For each user query, the agent first relies on **local RAG**:
    - It uses retrieve/advanced_retrieve (Qdrant + OpenAI embeddings, optionally Cohere rerank) to pull the most relevant chunks from data.txt and answer concrete salon questions (hours, services, durations, prices, and booking logic).
  - If the query is **not covered** by the local KB or explicitly about general hair information, the agent uses the **Tavily-powered search_web_current tool** to retrieve fresh web data.

- The LLM (via the LangGraph agent) decides which tool(s) to call, reads the tool outputs (KB chunks from Qdrant and/or Tavily web snippets), and then **composes a single natural-language answer**, always hiding raw tool outputs and exposing only the final, grounded response to the user.

Task 4: Build an end-to-end Agentic RAG application using a production-grade stack and your choice of commercial off-the-shelf model(s)

Deployed locally, code available at GitHub.

Task 5: Prepare a test data set (either by generating synthetic data or by assembling an existing dataset) to baseline an initial evaluation with RAGAS

SDG

In the evaluation.ipynb notebook, we synthetically generate a test dataset of customer queries for a hair salon by combining a hand-crafted domain document (data.txt) with three explicitly defined personas that capture different user behaviors (direct and efficient, indecisive and conversational, impatient returning client). Using Ragas' TestsetGenerator, we feed in the salon documentation and personas so that the model creates realistic, diverse user questions, along with their reference contexts and ground-truth answers, entirely from this synthetic domain description rather than from production logs. We then serialize this synthetic testset to JSON and use it to evaluate two retrieval-augmented agents (a baseline and an advanced variant) with Ragas metrics such as context recall, faithfulness, answer relevancy, and context precision. This setup allows us to systematically stress-test and compare agent behavior on realistic salon scenarios, while avoiding any use of real customer data and giving us fine-grained control over the types of interactions and edge cases represented in the evaluation.

Conclusions from the RAGAS evaluation (using `retrieve`)

- **Faithfulness**

- **Conclusion**: The pipeline is **highly faithful**, when it answers, it usually stays grounded in the retrieved context and the salon KB, with little hallucination.
- **Reasoning**: Our KB is small, factual, and business-specific (services, durations, prices, hours), so once the right chunk is retrieved, the model can answer directly without inventing extra details.

- **Context precision**
  - **Conclusion**: **Context precision is good but not perfect**: most retrieved chunks are relevant, but some queries still pull in extra or slightly off-topic chunks (e.g. neighboring services).
  - **Reasoning**: A basic retrieve over QdrantVectorStore with k=3 and character-level chunking tends to bring in what we need plus a bit of noise, this is acceptable for such a narrow domain, but leaves room for improvement via reranking (advanced_retrieve) or tighter chunking.
- **Context recall**
  - **Conclusion**: **Context recall is strong**. For the majority of evaluation questions, at least one of the retrieved chunks contains the key information needed to answer correctly (hours, durations, prices).
  - **Reasoning**: Because our KB is well-structured and not huge, dense retrieval with small k is enough to consistently surface the right chunk, especially for intent patterns like “how long does X take?” or “are you open on Y?”.
- **Answer relevancy**
  - **Conclusion**: **Answer relevancy is high**, the assistant’s final responses are usually on-topic and aligned with the user’s intent, particularly for booking, availability, and service questions.
  - **Reasoning**: Even when the retrieved context contains minor noise, the LLM can focus on the salient facts (duration, price, opening hours) and produce concise, user-focused answers, supported by the strong system prompt.

- **Overall effectiveness (retrieve-only pipeline)**
  - **Summary**: Our **retrieve-based RAG pipeline is effective and reliable for this salon use case**, because it retrieves the right facts most of the time, answers are grounded and relevant, and the model rarely contradicts the KB. The main trade-off is that without reranking (advanced_retrieve), we sometimes retrieve more context than necessary, so precision is slightly lower than it could be, but in a small, domain-specific KB, this doesn't significantly hurt answer quality.

	user_input	retrieved_contexts	response	reference	context_recall	faithfulness	answer_relevancy	context_precision
0	What are the business hours of the Hair Salon?	[# Hair Salon – Business Information ## Busine...	Hi — I can help with bookings, cancellations, ...	The Hair Salon is open Monday to Friday from 9...	1.0	0.000000	0.000000	1.0
1	How much for Women's Haircut?	[# Hair Salon – Business Information ## Busine...	Hi — welcome to the salon! How can I help toda...	Women's Haircut costs $35.	1.0	0.000000	0.000000	1.0
2	What is Balayage and how long does it take?	[# Hair Salon – Business Information ## Busine...	Hi — I can help schedule or cancel appointment...	Balayage is a hand-painted highlighting techni...	1.0	0.000000	0.747206	1.0
3	Wht is the price for Men's Haircut?	[# Hair Salon – Business Information ## Busine...	Hi — I'm your salon booking assistant. Would y...	The price for a Men's Haircut is $20.	1.0	0.000000	0.000000	1.0
4	Wht is the price for a blow-dry?	[# Hair Salon – Business Information ## Busine...	Hi — I can help with bookings, cancellations, ...	The price for a blow-dry is $25.	1.0	0.111111	0.000000	1.0
5	Can I change my appointment at Luna Hair Studio?	[💬 Pricing FAQs Do you offer student discounts...	Hi — I can help book or cancel appointments, o...	Yes, appointments at Luna Hair Studio can be r...	1.0	0.375000	0.771370	1.0
6	Do you offer student discounz at Luna Hair Stu...	[💬 Pricing FAQs Do you offer student discounts...	Hi — how can I help you today? I can book or c...	Occasional seasonal promotions may include stu...	1.0	0.571429	0.780088	1.0
7	Can I reschedule my appointment at Luna Hair S...	[💬 Pricing FAQs Do you offer student discounts...	Hi — I can help with bookings, cancellations, ...	Yes, appointments at Luna Hair Studio can be r...	1.0	0.555556	0.000000	1.0
8	Luna Hair Studio, do you have student discounts?	[💬 Pricing FAQs Do you offer student discounts...	Hi — I can help with that. Which service would...	Occasional seasonal promotions may include stu...	1.0	0.750000	0.000000	1.0
9	What are the policies regarding appointment ch...	[💬 Pricing FAQs Do you offer student discounts...	Hi — I'm the salon booking assistant. We're op...	At Luna Hair Studio, appointments can be resch...	1.0	0.300000	0.844546	1.0

Task 6: Install an advanced retriever of your choosing in our Agentic RAG application

Why the advanced retrieval is useful

- **Chosen technique**: We use an **advanced retrieval step that first fetches a wider set of candidates from Qdrant (higher k) and then applies** Cohere's rerank-v3.5 model **to re-score and keep only the top few chunks for the LLM.**
- **Why it helps this use case**: For the salon domain, many chunks look semantically similar (e.g. several services with prices and durations), so reranking by query–document relevance helps surface the *exact* service, time window, or policy the user asked about, which reduces noise and makes answers more precise.

Performance vs. original `retrieve` (RAGAS comparison)

Based on these RAGAS scores there's essentially **no meaningful difference** between the two.

- **Context recall & precision**: Both baseline and advanced retrieve the right passages perfectly (context_recall = 1.0, context_precision = 1.0), so the advanced retriever is not improving which chunks are found.
- **Faithfulness**: Baseline ≈ **0.2663**, advanced ≈ **0.2542** – the advanced variant is actually **slightly worse**, but the gap is tiny and likely within noise.
- **Answer relevancy**: Baseline ≈ **0.3143**, advanced ≈ **0.3161** – the advanced variant is **very slightly better**, but again the difference is negligible.

So, for this synthetic salon testset, our "advanced" retrieval setup does **not** show a clear improvement over the original, they behave almost identically according to RAGAS. Because the document set is relatively small, the difference in evaluation metrics is modest. With fewer documents, even a basic retriever performs reasonably well, so the impact of reranking and expanded retrieval becomes more noticeable only at larger scale.

	user_input	retrieved_contexts	response	reference	context_recall	faithfulness	answer_relevancy	context_precision
0	What are the business hours of the Hair Salon?	[# Hair Salon – Business Information ## Busine...	Hi — I'm the salon booking assistant. Would yo...	The Hair Salon is open Monday to Friday from 9...	1.0	0.000000	0.000000	1.0
1	How much for Women's Haircut?	[# Hair Salon – Business Information ## Busine...	Hi — how can I help today? Would you like to b...	Women's Haircut costs $35.	1.0	0.000000	0.000000	1.0
2	What is Balayage and how long does it take?	[# Hair Salon – Business Information ## Busine...	Hi — I can help with bookings, cancellations, ...	Balayage is a hand-painted highlighting techni...	1.0	0.375000	0.763613	1.0
3	Wht is the price for Men's Haircut?	[# Hair Salon – Business Information ## Busine...	Hi — we're open Mon–Fri 09:00–18:00, Sat 09:00...	The price for a Men's Haircut is $20.	1.0	0.750000	0.837273	1.0
4	Wht is the price for a blow-dry?	[# Hair Salon – Business Information ## Busine...	Hi — I can help schedule, change, or cancel ap...	The price for a blow-dry is $25.	1.0	0.000000	0.732757	1.0
5	Can I change my appointment at Luna Hair Studio?	[💬 Pricing FAQs Do you offer student discounts...	Hi — I'm the salon booking assistant. What ser...	Yes, appointments at Luna Hair Studio can be r...	1.0	0.000000	0.000000	1.0
6	Do you offer student discounz at Luna Hair Stu...	[💬 Pricing FAQs Do you offer student discounts...	Hi — I can help book, cancel, or answer questi...	Occasional seasonal promotions may include stu...	1.0	0.666667	0.000000	1.0
7	Can I reschedule my appointment at Luna Hair S...	[💬 Pricing FAQs Do you offer student discounts...	Hi — how can I help you today? Would you like ...	Yes, appointments at Luna Hair Studio can be r...	1.0	0.000000	0.000000	1.0
8	Luna Hair Studio, do you have student discounts?	[💬 Pricing FAQs Do you offer student discounts...	Hi — I'm your salon booking assistant. I can b...	Occasional seasonal promotions may include stu...	1.0	0.375000	0.000000	1.0
9	What are the policies regarding appointment ch...	[💬 Pricing FAQs Do you offer student discounts...	Hi — we're open Mon–Fri 9:00–18:00, Sat 9:00–1...	At Luna Hair Studio, appointments can be resch...	1.0	0.375000	0.827133	1.0

Task 7: Next Steps

I plan to **keep the RAG implementation based on dense vector retrieval for Demo Day**. The RAG setup itself is valuable, because it cleanly separates knowledge from the model, lets me update the salon documentation without retraining, and works well with my synthetically generated testset (high context recall and precision). However, the "advanced" retrieval logic does not show a meaningful improvement over the original in the RAGAS evaluation, so adding that extra complexity I am not sure is justified for the final

demo. Keeping the lean, dense-retrieval RAG pipeline gives me a clear, explainable architecture that’s robust enough for Demo Day while remaining easy to maintain and reason about.